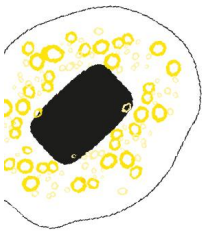
UNIVERSITY OF TWENTE.

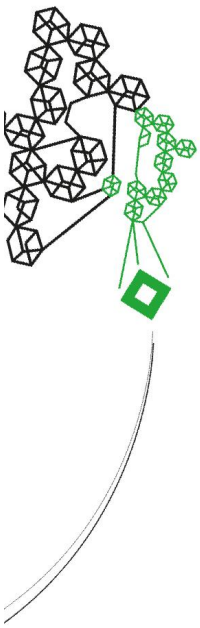


# MATLAB assignment
# - Code and Project Report

**2024-202400656-JAAR**

# *Overlapped Spheres*

*Oliwier Piotr Sobczyk – s2597047*
*o.p.sobczyk@student.utwente.nl*



Igor Ostanin
i.ostanin@utwente.nl
University of Twente

# 1 Introduction

This report concluded the 2024-202400656-JAAR Matlab learning path. This undertaking aimed to develop a function that addresses a given question utilizing the skills and insights acquired during the course. The selected task involved crafting a function determining whether spheres overlap in a three-dimensional space.

Task: Utilize Matlab to determine overlaps among spheres defined by coordinates and radii. The function should accept a list of spheres, each with specific coordinates and radii, and return a list identifying pairs of intersecting spheres.

During the process, the following subtasks were introduced:

- read the data from xls file
- create a sphere structure containing center coordinates and radius
- generate N number of spheres with random coordinates and random radii
- display the results in figure
- save the results as indices pairs in a txt file

To introduce clarity to this task, several smaller function files were made:

- main.m - user interacts with this file
- sphereGen.m - function to generate spheres
- sphereStruc.m - function to make a sphere structure
- findOverlapSpheres.m - function which generates list of overlap sphere pairs
- checkOverlap.m - function called by the latter to verify sufficient distance between spheres
- svDisp.m - function to save results in txt

## 1.1 main.m

Upon executing the file, the user must **input** one: 'xls'/'predef'/'rand'. For the 'rand' option, an additional **input** of the number of spheres (integer) is required. Once the script calculates sphere positions and identifies overlaps, the user inputs (y/n) to determine whether the results will be viewed and saved. If affirmative, the **output** includes a text file listing overlapped sphere pairs, along with a plot featuring blue dots for non-overlapping spheres and red dots for overlapping ones.

## 1.2 sphereGen.m

**spheres = sphereGen(choice)**
**INPUT** is a string ('xls'/'predef'/'rand') or a list of list of spheres in form [x1,y1,z1,r1;...;xn,yn,zn,rn].
**OUTPUT** spheres structure with fields center and radius containing x,y,z coordinates and radius of each sphere

## 1.3 sphereStruc.m

**sphere = sphereStruc(center, radius)**
**INPUT** center is an array with [x,y,z] coordinates and radius is a double with radius of the current sphere
**OUTPUT** sphere structure with fields 'center' and 'radius' This function is useful when creating a sphere in the following form: sphereStruc([x,y,z],R)

### 1.4 findOverlapSpheres.m

**[overlapSph,spheres] = findOverlapSpheres(spheres)**
Main function used in this script. It can also be called from the command window by [overlapSph, spheres] = findOverlapSpheres([x1,y1,z1,r1;...;xn,yn,zn,rn])
**INPUT** is presented as a list formatted as [x1,y1,z1,r1;...;xn,yn,zn,rn] or as a sphere with center and radius attributes for each sphere. The function identifies whether it is dealing with a list or a structure, and for the latter, it generates the structure by invoking the sphereGen.m function.
**OUTPUT** is the list of pairs of overlapping spheres and sphere structures contacting each sphere's fields 'center' and 'radius'.

### 1.5 checkOverlap.m

**isOverlap = checkOverlap(sph1, sph2)** The function calculates the distance between two specified spheres by utilizing the norm function to determine their Euclidean distance. It then compares this distance to the sum of the spheres' radii to ascertain whether they intersect.
**INPUT** two sphere structures with fields 'center' and 'radius'.
**OUTPUT** boolean, True If spheres overlap, distance is less or equal to the sum of their radii, or false if otherwise

### 1.6 svDisp.m

**svDisp(overlapSph, spheres)** This function generates a three-dimensional plot depicting the results, where red and blue dots represent spheres that do and do not overlap. Additionally, a text file is produced listing the pairs of spheres that overlap.
**INPUT** overlapSph - list with overlapping sphere pairs, spheres - structure containing fields 'center' and 'radius' of all defined spheres
**OUTPUT** There is no output, but a figure and a text file are created.

## 2 Validation Examples

### 2.1 Predef

After running the file, the user should type 'predef' in the command window. The result should be:

```
Specify the set of spheres (type: xls/predef/rand): predef
There are 4 spheres which overlap. Overlapping Sphere Pairs (by Index):
     1     5
     4     5
     5     6

Do you wish to save and display results y/n?: n
Thank you for using this script
```

Figure 1: Result for the 'choice'

### 2.2 Xls file

Another choice of validation is to type 'xls' instead, then the result should be:

```
No string characters found in the array.
There are 16 spheres which overlap. Overlapping Sphere Pairs (by Index):
     2    14
     4    20
    10    43
    13    29
    16    42
    32    46
    33    48
    35    44

Do you wish to save and display results y/n?: y
```

Figure 2: Results for the 'xls' choice

### 2.3 Random spheres

Otherwise, the user can also choose 'rand', followed by the number of random spheres. It is possible to change the predefined sphere coordinates and radii in the sphereGen.m file or in the sphereDef.xls file, where the first three columns are x,y,z coordinates, respectively, and the fourth column is their radii.

### 2.4 Call from command window

The function findOverlapSpheres can also be called from the command window with the following command or in the form described in section 1.4: **[overlapSph,spheres] = findOverlapSpheres([5, 5, "st",100;3, 4, 21,5])**. Here the character array is inserted intentionally to present that the function recognises invalid input and converts it while running.

## 3 Discussion

In summary, the entire project incorporates various elements covered in the course. By breaking it into smaller sub-tasks, modularity was introduced, facilitating easier comprehension, quicker error detection and seamless integration of additional subfunctions. Several errors and preventions were considered to allow for a single run without errors.